Precision Under Fire: Analysis and Predictive Modeling in Counter-Strike: Global Offensive

Xinyao Xia*, Abel Salinas*, Fred Morstatter*
*USC Information Sciences Institute, Marina del Rey, California, USA
Email: {andyxia, asalinas, fred}@isi.edu

***Abstract*—Esports analytics has gained traction in recent years, leveraging machine learning (ML) to predict in-game events and enhance strategic decision making. This study develops structured datasets from Counter-Strike: Global Offensive (CS: GO) competitive matches to address two predictive tasks: round result prediction and player death prediction. Using data from professional *Dust 2* matches in 2022, we extract key player- and team-level features such as health, spatial positioning, and economy-related metrics. Various ML models, including Logistic Regression, Decision Trees, and XGBoost, are evaluated and benchmarked against random guessing and majority-class baselines. Results show that XGBoost consistently outperforms other models, effectively capturing gameplay dynamics and providing accurate predictions. These findings offer valuable insights for esports strategy optimization, coaching, real-time decision support, and applications in live-game analysis and betting. The dataset and methodology also establish a foundation for future esports research and predictive modeling across other game environments.**

***Index Terms*—Esports analytics, Predictive modeling, Machine learning, Gameplay dynamics.**

I. Introduction

Competitive esports generate vast amounts of gameplay data, which makes them an ideal domain for machine learning (ML) applications in predictive modeling. As professional matches involve highly strategic decision-making, team coordination, and economic management, understanding which factors influence round outcomes and player survival is critical for both competitive analysis and AI-driven decision support systems. Despite the availability of raw gameplay data, structured datasets tailored for predictive modeling remain limited, and existing studies often lack comprehensive feature engineering that captures both individual and team-level dynamics.

To address this, we built structured datasets from CS: GO competitive matches, focusing on round result prediction and player death prediction. This study selects *Dust 2*, one of the most played and strategically balanced maps, as a benchmark for analyzing player behavior and engagement patterns. By capturing spatial, economic, and combat-related features, these datasets provide a foundation for evaluating ML models on in-game event prediction.

We evaluated multiple ML models, benchmarking them against random guessing and majority class baselines, to assess how well player actions, team strategies, and game mechanics influence match outcomes. The findings contribute to esports analytics and have practical applications in coaching, strategy optimization, and real-time decision support systems.

II. Related Work

Predicting outcomes and in-game events in esports has garnered significant interest in recent years due to its potential to enhance audience engagement, strategic gameplay analysis, and automated coaching systems. Several studies have explored similar domains, focusing on diverse aspects of predictive modeling in esports. [1] addressed the challenges of preprocessing complex esports data for downstream analysis, particularly in CS: GO. Their methodology for data parsing and player matching provides foundational insights into handling large, undocumented datasets—a process central to this study. [2] introduced a valuable data model for extracting data from CS: GO's proprietary replay data files, enabling new research collection of granular in-game data for predictive analytics. [3] examined the feasibility of predicting CS: GO match outcomes by analyzing team compositions using ML. [4] introduced a game-level win probability model to assess team spending decisions. Their study identifies suboptimal purchasing strategies and proposes the Optimal Spending Error (OSE) metric to evaluate how closely teams align their spending with predicted optimal decisions. [5] explored round and game-winner prediction in CS: GO, utilizing player purchasing decisions and match performance metrics as key predictive features. Their study evaluates the effectiveness of Random Forest, XGBoost, and Multi-layer Perceptron models, finding that Random Forest performed best for round-level predictions, while XGBoost excelled at predicting game outcomes. [6] applied machine learning models to round result prediction in CS: GO, incorporating *TrueSkill* ratings to improve accuracy, where *TrueSkill* is a Bayesian ranking system designed to estimate player skill dynamically based on past performances. [7] developed a real-time round winner prediction model using game-state data and Markov Chains, dynamically updating round-winning probabilities based on in-game events. [8] demonstrated the use of neural networks to predict in-game deaths within a three-second window in Counter-Strike: Global Offensive (CS: GO). Their results laid the groundwork for predictions of death events. Beyond CS: GO, [9] explored micro-predictions of death events in Dota 2, demonstrating the potential for deep learning to accurately predict rare events within complex game environments. Similarly, [10] applied LSTMs to real-time outcome prediction in competitive video games, underscoring the applicability of sequence modeling for esports data. These works collectively highlight the grow-

2025 IEEE Conference on Games (CoG) | 979-8-3315-8904-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/COG64752.2025.11114164

Authorized licensed use limited to: Vilnius University. Downloaded on October 22,2026 at 16:07:46 UTC from IEEE Xplore. Restrictions apply.

ing capabilities of ML and data analysis in esports, providing a foundation for our predictive modeling tasks: round result prediction and player death prediction in CS: GO. While prior research primarily emphasizes match-level and round-level predictions, our study expands this by incorporating both round-level and player-level modeling, capturing individual decision-making, positional information, and in-game economic conditions. By building upon these studies, our research improves predictive analytics in esports by modeling in-game events at multiple levels of analysis with a more granular approach.

III. Curating a CS: GO Dataset

To ensure relevance and consistency in gameplay data, this study uses CS: GO tournament matches from 2022, the final full year of competitive CS: GO before the release of Counter-Strike 2 in September 2023. This study focuses exclusively on the map *Dust 2*, a strategically balanced and widely played map within Bomb Defusal mode. The right side of Figure 1 displays the map, while the left side presents a heatmap of all player deaths. The constraints of focusing on a single map enhance consistency and reduce variability in the dataset. In Bomb Defusal, two teams—Terrorists (T) and Counter-Terrorists (CT)—compete with distinct objectives: the Terrorists must plant and defend the bomb, while the Counter-Terrorists aim to prevent the plant or defuse it if planted and ensure it explodes, while the Counter-Terrorists aim to prevent the bomb from being planted or to defuse it if it is planted. A round can also be won by eliminating all opponents, but if the bomb is planted, it must still be defused to secure a CT victory. Matches are played in a 5v5 format, with up to 30 rounds, and teams switch sides after 15 rounds. The first team to win 16 rounds secures the match; otherwise, the game goes to overtime. Each round is time-limited. Strategic teamwork, in-depth knowledge of the map's layout, and efficient resource management are crucial in securing the victory for either side.

All the CS: GO tournament data is sourced from HLTV,[1] a widely recognized platform for competitive CS: GO statistics. Match data is obtained using an automatic demo downloader[2]. The obtained data serves as recordings of all in-game events and is stored in a proprietary format with a `.dem` extension, referred to as a "demo file." This study uses the `AWPY` python library created by [2] to parse demo files and uses the tools from the library to extract feature information for the predictive task.

This study explores two key prediction tasks in CS: GO, forecasting round results and predicting player deaths within given time windows. Round result prediction seeks to determine whether the Terrorists or Counter-Terrorists will win a given round, using in-game factors such as player health, weapon choices, economic conditions, and remaining teammates at the moment the bomb is planted. This task is crucial for understanding how team composition, engagement dynamics, and strategy influence round outcomes, offering insights into the decision-making process at a pivotal moment in the game.

Player death prediction focuses on whether a specific player will die within a given time frame, capturing individual engagement risk based on real-time factors. Instead of relying solely on team-wide statistics, this task evaluates player positioning, weaponry, health status, and opponent proximity to determine the likelihood of survival. By predicting deaths within 3 different time windows, this task provides a deeper understanding of individual player vulnerabilities and tactical decision-making in engagements.

A. Data Collection

To address the two prediction tasks, the data was split into two separate datasets: one for round result prediction and another for player death prediction. While both datasets originate from the same source—all *Dust 2* games in all 1,444 official CS: GO tournaments listed on HLTV in 2022-they differ in parsing methods, features, and structures. Some tournament files were corrupted, and after processing, 15,589 rounds of gameplay were successfully parsed (Figure 2).

1) Round Result Prediction Dataset: To collect this dataset, the parsing process tracks all key events, such as kills, damage, and bomb plants, providing a comprehensive view of the round. The target variable is binary, indicating whether Terrorists win the round (`T_Wins`). As shown in Table 2a, this dataset includes all 15,589 rounds with 11,606 positive instances and 3,983 negative instances.

2) Player Death Prediction Dataset: The dataset is structured into three subsets based on different time windows (5s, 10s, 15s) before a potential death event. These windows were selected to capture distinct tactical horizons relevant to in-game decision-making: 5 seconds reflects imminent danger, requiring fast reflexes; 10 seconds represents a medium-term engagement phase; and 15 seconds provides context for longer-term strategies or ambushes. Player stats are recorded at a randomly selected time between 3 to 15 seconds before death, ensuring variability, diversity, and uniform distribution. The target variable is whether the player will die within the timeframe (`will_die`). For true labels, player stats are collected based on this randomly chosen time window. For false labels, player stats are recorded by ensuring that the selected player does not die within the corresponding time window. Each subset captures detailed player-level attributes, including health, armor, weapon types, position, and in-game states (e.g., scoped, ducking). By structuring the dataset into multiple survival time frames while retaining a randomized collection process, we improve the ability to model different decision-making horizons, from immediate threats (5-second subset) to longer-term engagements (15-second subset). Since longer time windows inherently encompass shorter ones, the dataset sizes for the 10-second and 15-second time windows are approximately twice and three times the size of the 5-second time window dataset, respectively. To mitigate potential bias caused by this disparity, we employed the `sample()`

[1]https://HLTV.org

[2]https://c9win.com/demo-downloader/

Authorized licensed use limited to: Vilnius University. Downloaded on October 22,2026 at 16:07:46 UTC from IEEE Xplore. Restrictions apply.

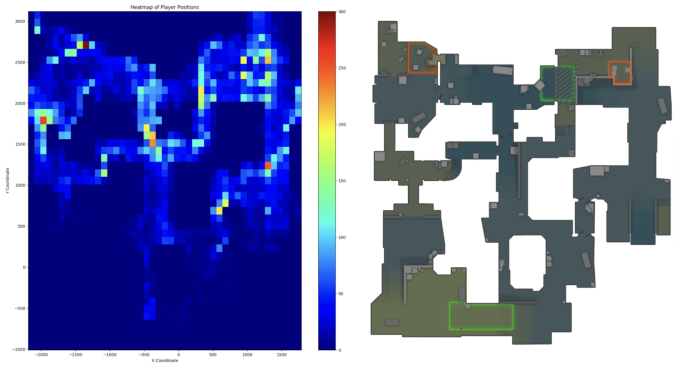


Fig. 1: Heatmap depicting the distribution of death locations in Dust 2 (left), with a depiction of the map (right).

(a) Round Result Prediction Dataset

Measure	Count
Demo Files	1,444
Rounds	15,589
Datapoints	15,589
Positive Class	11,606
Negative Class	3,983

(b) Player Death Prediction Dataset (Within 5 Seconds)

Measure	Count
Demo Files	1,444
Rounds	15,589
Datapoints	83,232
Positive Class	41,267
Negative Class	41,965

(c) Player Death Prediction Dataset (Within 10 Seconds)

Measure	Count
Demo Files	1,444
Rounds	15,589
Datapoints	83,232
Positive Class	41,227
Negative Class	42,005

(d) Player Death Prediction Dataset (Within 15 Seconds)

Measure	Count
Demo Files	1,444
Rounds	15,589
Datapoints	83,226
Positive Class	41,155
Negative Class	42,071

Fig. 2: Number of observations for round result and player death datasets.

function in Python to downsample the 10-second and 15-second datasets to match the sample size of the 5-second dataset, while ensuring class balance. As shown in Tables 2b, 2c, and 2d, all three datasets contain approximately 83,232 instances, with balanced positive and negative labels.

B. Feature Generation

Features of both datasets are summarized in Figure I, reflecting key aspects of player and team-level dynamics. All parsed data consists of individual player statistics recorded at specific time points. For round result prediction, data is collected at the moment of the bomb plant, capturing the game state at a pivotal decision point. For player death prediction, data is recorded at a randomly selected time window to ensure variability while preserving real-time player states. Feature generation involves aggregating player and team-level statistics to capture both individual and team dynamics.

Team-level features provide a macro-level view of overall team dynamics by incorporating variables such as `teamDistance` and `teamEqVal`. `TeamDistance` represents the average distance of all players on a team to the bombsite if the bomb is planted or the nearest bombsite if the bomb has not been planted, which serves as an indicator of map control and strategic positioning—teams closer to the bombsite may be in a better position to execute or defend, while those farther away may face a disadvantage in retake or site control scenarios. `teamEqVal` reflects the total equipment and weapon value of a team, an essential factor indicating how much damage the team can potentially cause. This factor closely relate to combat effectiveness and the likelihood of winning engagements.

Player-level data inherits the original parsed data and it also has aggregated data for each player which includes `total_kills`, `total_deaths`, and `average_damage_per_round` , along with other performance metrics. In the player death prediction dataset, some team-level statistics such as `average_teammate_distance` and `average_teammate_health` are also recorded. Both statistics measure the average Euclidean distance from teammates and the average health point of teammates who are alive, which are indicators of available backups and fire

Authorized licensed use limited to: Vilnius University. Downloaded on October 22,2026 at 16:07:46 UTC from IEEE Xplore. Restrictions apply.

support from teammates. These features correlate to whether the player can either win the engagement or escape from a threatening engagement.

Opponent-related statistics are also included in this dataset to encompass more diverse in-game dynamics. `average_enemy_distance`, `average_enemy_health`, `hp_diff_with_enemy`, `num_enemies_ahead`, and `econ_disparity` capture the comparison between the player and enemies from spatial and economic perspectives. These features provide insights into engagement risks and tactical positioning. For instance, `average_enemy_distance` assesses how proximity to opponents influences survival likelihood. `Econ_disparity` reflects the relative strength of a player's team's economy compared to their opponents, which largely determines the survival rates of the player when engagement happens as higher equipment values indicate higher damages and higher armor values indicate higher limits of damages the player is able to survive. `Num_enemies_ahead` measures the number of enemies in the direction where the player is moving. It measures direct risk posed by opponents as the player may or may not be aware of enemies ahead. By incorporating these opponent-related features, the dataset enables models to evaluate how player survival and decision-making are influenced by enemy positioning, health advantages, and economic conditions.

The heatmap in Figure 1 provides additional context by visualizing high-risk engagement areas. It highlights locations where player deaths frequently occur. The two orange-boxed regions represent Bombsite A and Bombsite B, which, as shown on the heatmap, are key engagement zones where a significant number of deaths take place. This visualization reinforces the importance of spatial positioning with teammates and opponents in CS: GO, aligning with the dataset's distance-based features in capturing tactical and engagement dynamics.

IV. Experiments

A. Models

To evaluate the predictive capability of the dataset, this study applies multiple machine learning models, including Logistic Regression, Decision Tree Classifier, and Extreme Gradient Boosting(XGBoost). These models are selected to demonstrate the predictive nature of simple models and maintain interpretability and transparency. Preprocessing steps include normalization, one-hot encoding, imputation for missing values, and reducing false label class size, ensuring readiness for predictive modeling. The results of these models are compared and evaluated. Each model is fed with the same processed data as input to compare the performance.

Here are the models applied:

- **Logistic Regression**: a probabilistic model generally used for classification tasks. Logistic regression aims to determine the influence of independent variables on the dichotomous dependent variable.
- **Decision Tree Classifier**: an ML algorithm that classifies data by splitting it into subsets based on feature values, forming a tree structure. Each split represents a decision, leading to final class labels at the leaf nodes. [11]
- **XGBoost**: an advanced ML algorithm based on gradient-boosted decision trees. It builds models sequentially, where each new tree corrects errors from the previous ones, resulting in high accuracy and resilience to overfitting. XGBoost is highly efficient, can handle large datasets, and includes features like regularization and parallel processing, making it well-suited for complex classification and regression tasks. [12]

To ensure robustness, we applied 5-fold stratified cross-validation, where each model was trained and tested on different subsets of the data while maintaining the original class distribution. Logistic Regression and Decision Tree Classifier are implemented through *Scikit-learn* [13], a commonly used Python package for expressing and executing ML algorithms. XGBoost is implemented through *xgboost* library, a Python package for efficient gradient boosting, designed to enhance performance in ML tasks [12].

B. Hyperparameter Tuning

To optimize model performance while ensuring balanced predictions across both classes, we performed hyperparameter tuning using Bayesian Optimization (*Optuna*) and *Grid Search*. We used a Python library called *Optuna* to implement Bayesian Optimization [14] and `GridSearchCV` from *Scikit-learn* to implement *Grid Search*. The tuning process aimed to maximize macro-F1 score while employing 5-fold stratified cross-validation to prevent overfitting and ensure robust evaluation. The choice of optimization methods was guided by the complexity of the search space, computational efficiency, and the nature of each model.

XGBoost was optimized using Bayesian Optimization via Optuna, tuning key hyperparameters to balance predictive accuracy and generalization. The most influential were learning rate, max depth, and number of estimators, while subsample, colsample_bytree, and L1/L2 regularization controlled overfitting and model stability. For round result prediction, the optimal configuration used a shallow tree (max depth = 3) with 139 estimators and a learning rate of 0.102, indicating reliance on a few key features. A subsample of 0.771 and colsample_bytree of 0.607 maintained stability, while moderate L1 (0.072) and L2 (3.208) regularization prevented overfitting. For player death prediction, deeper trees were required. The 5-second model used max depth = 5, 752 estimators, and a low learning rate (0.023) for fine-grained decision-making, with strong L1 regularization (12.91) for complexity control. The 10-second model increased max depth to 7 with 561 estimators, reducing L1 (3.70) and nearly eliminating L2 (0.0001), prioritizing dynamic in-game actions. The 15-second model decreased max depth back to 5 with 613 estimators and a higher learning rate (0.073), shifting toward broader engagement patterns with stronger L2 regularization (69.51) for long-term generalization.

Logistic Regression was optimized using *Grid Search* to tune regularization strength (`C`), solver choice, and the

Authorized licensed use limited to: Vilnius University. Downloaded on October 22,2026 at 16:07:46 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Feature descriptions for engineered features for predictive tasks.

(a) Round Result Prediction Features

Feature	Explanation
T_alives	Number of Terrorists alive
T_teamHP	Total health points of Terrorists
T_teamARMOR	Total armor points of Terrorists
T_teamEqVal	Total equipment value of Terrorists
T_teamCash	Total cash available for spending for Terrorists
T_teamCashSpent	Total cash spent during this round for Terrorists
T_teamDamage	Total damage Terrorists has caused to Counter-Terrorists during the round
T_totalKills	Total kills of Terrorists into the match
T_roundKills	Total kills of Terrorists into the round
T_totalDeaths	Total deaths of Terrorists into the match
T_teamAveDamage	Average amount of damage output from a Terrorist across all previous rounds in the match
T_teamDistance	Average distance between Terrorists and the bomb site
T_primary_weapon	Weapon chosen from primary weapon category(One-Hot-Encoder)
T_secondary_weapon	Weapon chosen from secondary weapon category(One-Hot-Encoder)
T_grenades	Average number of grenades held by Terrorists
T_flashes	Average number of flash grenades held by Terrorists
T_smokes	Average number of smoke grenades held by Terrorists
T_molotovs	Average number of molotovs held by Terrorists
CT_alives	Number of Counter-Terrorists alive
CT_teamHP	Total health points of Counter-Terrorists
CT_teamARMOR	Total armor points of Counter-Terrorists
CT_teamEqVal	Total equipment value of Counter-Terrorists
CT_teamCash	Total cash available for spending for Counter-Terrorists
CT_teamCashSpent	Total cash spent during this round for Counter-Terrorists
CT_teamDamage	Total damage Counter-Terrorists has caused to Terrorists during the round
CT_totalKills	Total kills of Counter-Terrorists into the match
CT_roundKills	Total kills of Counter-Terrorists into the round
CT_totalDeaths	Total deaths of Counter-Terrorists into the match
CT_teamAveDamage	Average amount of damage output from a Counter-Terrorist across all previous rounds in the match
CT_teamDistance	Average distance between Counter-Terrorists and the bomb site
CT_primary_weapon	Weapon chosen from primary weapon category(One-Hot-Encoder)
CT_secondary_weapon	Weapon chosen from secondary weapon category(One-Hot-Encoder)
CT_grenades	Average number of grenades held by Counter-Terrorists
CT_flashes	Average number of flash grenades held by Counter-Terrorists
CT_smokes	Average number of smoke grenades held by Counter-Terrorists
CT_molotovs	Average number of molotovs held by Counter-Terrorists
bombSite_A	Location of where the bomb is planted (A/B)

(b) Player Death Prediction Features

Feature	Explanation
hp	Health points of the player
armor	Armor points of the player
equipment_value	Total monetary value of all the gear and weapons the player possesses
cash	Cash available for spending
cash_spend_this_round	Cash spent during the round
round_damage	Total damage the player has caused to other players during the round
primary_weapon	Weapon chosen from primary weapon category
secondary_weapon	Weapon chosen from secondary weapon category
active_weapon_category	Weapon currently selected by the player
grenades_left	Number of grenades held by player
flashes_left	Number of flashes held by player
smokes_left	Number of smokes held by player
molotovs_left	Number of molotovs held by player
is_blinded	Whether the player is blinded by a flashbang grenade
is_ducking	Whether the player is crouching(ducking)
is_scoped	Whether the player is currently looking through the scope of a weapon
has_helmet	Whether the player has a helmet
has_defuser	Whether the player has a defuser to defuse the bomb
total_kills	Total kills so far into the match
round_kills	Total kills so far into the round
total_deaths	Total deaths so far into the match
callout	Location of the player on the map
average_damage_per_round	Average amount of damage output from a player across all previous rounds in the match
average_teammate_distance	Average euclidean distance from teammates
average_enemy_distance	Average euclidean distance from enemies
avg_teammate_health	Average health points of teammates that are alive
avg_enemy_health	Average health points of enemies that are alive
econ_disparity	Difference between total equipment value of the team and the total equipment value of the enemy team
hp_diff_with_enemy	Difference between the player's health points and the closest enemy's health points
num_enemies_ahead	Number of enemies at the rough direction where the player is moving

	Default Parameters			Tuned Parameters		
Model	Accuracy	F1 (Class 0)	F1 (Class 1)	Accuracy	F1 (Class 0)	F1 (Class 1)
Logistic Regression	0.858	0.702	0.907	0.859	0.703	0.908
Decision Tree	0.797	0.606	0.863	0.841	0.656	0.896
XGBoost	0.850	0.690	0.901	0.859	0.709	0.907
Random Guess	0.500	0.500	0.500	0.500	0.500	0.500
Majority Class	0.740	0.000	0.850	0.740	0.000	0.850

TABLE II: Performance comparison for round result prediction between default models and tuned models.

number of iterations. Regularization is critical for preventing overfitting, with higher `C` values reducing regularization and lower `C` values enforcing stronger constraints on coefficients. The solver determines the optimization algorithm, with `liblinear` suited for smaller datasets and `lbfgs` handling larger-scale problems. For round result prediction,

Authorized licensed use limited to: Vilnius University. Downloaded on October 22,2026 at 16:07:46 UTC from IEEE Xplore. Restrictions apply.

Model	Default Parameters			Tuned Parameters		
	Accuracy	F1(Class 0)	F1(Class 1)	Accuracy	F1(Class 0)	F1(Class 1)
Logistic Regression	0.654	0.659	0.649	0.655	0.660	0.649
Decision Tree	0.583	0.588	0.579	0.644	0.659	0.628
XGBoost	0.664	0.673	0.655	0.672	0.682	0.661
Random Guess	0.500	0.500	0.500	0.500	0.500	0.500
Majority Class	0.504	0.670	0.000	0.504	0.670	0.000

TABLE III: Performance comparison for player death prediction between default models and tuned models (within 5 seconds).

Model	Default Parameters			Tuned Parameters		
	Accuracy	F1(Class 0)	F1(Class 1)	Accuracy	F1(Class 0)	F1(Class 1)
Logistic Regression	0.649	0.655	0.643	0.649	0.655	0.643
Decision Tree	0.578	0.582	0.574	0.637	0.654	0.616
XGBoost	0.656	0.664	0.649	0.667	0.678	0.655
Random Guess	0.500	0.500	0.500	0.500	0.500	0.500
Majority Class	0.505	0.671	0.000	0.505	0.671	0.000

TABLE IV: Performance comparison for player death prediction between default models and tuned models (within 10 seconds).

Model	Default Parameters			Tuned Parameters		
	Accuracy	F1(Class 0)	F1(Class 1)	Accuracy	F1(Class 0)	F1(Class 1)
Logistic Regression	0.646	0.653	0.639	0.647	0.653	0.640
Decision Tree	0.578	0.584	0.572	0.635	0.649	0.621
XGBoost	0.653	0.662	0.644	0.662	0.673	0.650
Random Guess	0.500	0.500	0.500	0.500	0.500	0.500
Majority Class	0.506	0.671	0.000	0.506	0.671	0.000

TABLE V: Performance comparison for player death prediction between default models and tuned models (within 15 seconds).

Works	Model	Accuracy
xu2022predicting	XGBoost	0.792
Rubin2022	Random Forest	0.641
This paper	XGBoost	0.859

TABLE VI: Prior Work Comparison for Round Result

Works	Model	F1 (Class 1)
marshall2022enabling	LSTM	0.38
This paper	XGBoost	0.661

TABLE VII: Prior Work Comparison for Player Death

the best configuration used `C` = 100 with L2 regularization and `liblinear` solver, suggesting that a weaker regularization constraint provided the best fit for the dataset. For player death prediction, the optimal `C` values varied across time windows, reflecting differences in model complexity and generalization needs. The 5-second model performed best with `C` = 0.070, keeping strong regularization to avoid overfitting, using `liblinear` solver. The 10-second model further decreased `C` to 0.034, suggesting that mid-range survival prediction benefits from even stronger constraints, requiring `lbfgs` solver for better optimization. In contrast, the 15-second model increased `C` to 0.616 with the same solver, relaxing regularization constraints as longer-term predictions became more dependent on diverse game states.

Decision Trees were optimized using *Grid Search*, tuning max depth, splitting `criterion`, and minimum sample constraints (`min_samples_split`, `min_samples_leaf`) to balance complexity and generalization. For round result prediction, the optimal model used Gini impurity, max depth of 5, and minimal constraints (`min_samples_split` = 2, `min_samples_leaf` = 1), suggesting that a shallow tree efficiently captured round dynamics. For player death prediction, deeper trees with entropy-based splitting were preferred, reflecting the higher complexity of individual survival outcomes. The 5-second model required max depth of 11 and stricter constraints (`min_samples_split` = 41, `min_samples_leaf` = 5) to prevent overfitting. The 10-second model used max depth of 9 with slightly stronger regularization (`min_samples_leaf` = 9), while the 15-second model maintained a depth of 9 but further increased `min_samples_leaf` to 19 for stability.

C. Evaluation Method

Accuracy and F1-score are used to evaluate each model's performance and misclassification errors. Accuracy measures the proportion of correct predictions across both classes, providing an overall assessment of model performance. F1-score accounts for the balance between precision and recall, making it particularly useful in ensuring that the model performs well on both classes rather than favoring the majority class. By evaluating F1-scores for each class separately, we ensure that the model does not disproportionately misclassify instances from the minority class. High accuracy and F1-scores suggest that the dataset captures meaningful patterns and provides sufficient diversity for the models to generalize effectively to unseen data.

Authorized licensed use limited to: Vilnius University. Downloaded on October 22,2026 at 16:07:46 UTC from IEEE Xplore. Restrictions apply.

D. Results

To evaluate model performance, accuracy and F1-scores are compared for both classes across round result prediction and player death prediction. Additionally, random guessing and majority class serve as benchmarks to assess whether the trained models provide meaningful predictive capability.

Table II presents the round result prediction results, comparing models with default and tuned hyperparameters. Logistic Regression performs best with 0.858 accuracy and 0.907 F1-score (Class 1), followed by XGBoost (0.85 accuracy, 0.901 F1-score). Decision Trees perform slightly worse (0.797 accuracy, 0.863 F1-score). Hyperparameter tuning yields minor improvements, with XGBoost reaching 0.859 accuracy and 0.907 F1-score, while Decision Trees improve in F1 but lose slight accuracy. The random guessing baseline (0.50 accuracy, 0.50 F1-score) and majority class baseline (0.74 accuracy, 0.85 F1-score) confirm that trained models offer meaningful classification advantages.

Tables III, IV, and V present player death prediction results across 5s, 10s, and 15s time windows. XGBoost consistently performs best, peaking at 0.664 accuracy and 0.673 F1-score (Class 0) in the 5s window. Decision Trees show the most variability, while some models exceed the majority class baseline (F1 = 0.671 for Class 0) in accuracy but not F1-score, suggesting over-prediction of the majority class. Threshold tuning may be needed to better balance precision and recall for Class 0. Hyperparameter tuning significantly improves Decision Trees across all time windows and helps XGBoost surpass the majority class baseline in F1-score.

Table VI compares round result prediction with prior works by [6] and [5]. Our XGBoost model outperforms both, indicating stronger predictive signals in our dataset. For player death prediction, the closest comparison is [8], which employs a 3-second prediction window using a Long Short-Term Memory (LSTM) model. As shown in Table VII, [8] achieve an F1-score of 0.38 for Class 1 (player will die within 3 seconds), whereas our XGBoost model, evaluated on a 5-second window, achieves an F1-score of 0.661, demonstrating a substantial improvement. This suggests that structured feature engineering and tree-based models offer a more effective predictive framework for short-term in-game events.

XGBoost's superior performance is likely due to its ability to capture complex feature interactions and handle nonlinear relationships better than Logistic Regression and Decision Trees. Unlike single-tree models, XGBoost aggregates multiple decision trees, reducing variance while maintaining interpretability. The performance gap between our models and prior works can be attributed to dataset quality, feature selection, and modeling approaches. Our dataset includes richer predictive features, such as spatial positioning, economic factors, and teammate/enemy proximity, which were not fully explored in previous studies.

V. Limitations

This study focuses exclusively on the *Dust 2* map. While the core game mechanics–such as player movement, economic management, and weapon strategies–are consistent across maps, map-specific spatial factors (e.g., chokepoints, sightlines, engagement frequency) may influence the learned patterns. To better understand the generalizability of the models to other maps, future work will be to expand the dataset to include multiple maps. This will help disentangle general gameplay behaviors from map-dependent influences, improving the robustness of the findings.

In addition, other potential features could be examined to improve prediction accuracy and generalizability. [15] explored how personality could impact gaming styles. Since tournament player statistics are public, incorporating player-specific behavioral traits could improve predictions, particularly in high-pressure situations like clutch scenarios or retakes, where aggression or passivity impacts survival.

Moreover, this study focuses on tournament matches, which feature structured team coordination and strategic play. The models' applicability to casual or ranked games remains uncertain due to differences in team dynamics and skill levels. Expanding to broader datasets could assess whether the models hold predictive value across diverse playstyles.

While deep learning models like LSTMs or Graph Neural Networks could have been used, this study prioritizes interpretable, efficient models to demonstrate the predictive power of structured gameplay data. Future work could explore more complex architectures, but the findings highlight that even lightweight ML models can extract meaningful insights from CS: GO match data.

VI. Dataset Significance and Implications

This dataset provides a structured and detailed representation of in-game player states, economic conditions, and situational factors in CS: GO competitive matches and allows for predictive modeling of both round results and player deaths. By incorporating features such as player health, equipment value, spatial positioning, and economy-related metrics, it captures the key elements that influence in-game decision-making. The structured format enables machine learning models to assess player survival and round-winning probabilities, which offers a data-driven approach to analyzing tactical outcomes in professional play.

Beyond model performance evaluation, this dataset has practical implications for players, coaches, and analysts. The ability to predict player deaths within specific time windows could be applied to training and decision-support tools and provide early-warning indicators based on real-time engagement risks. Coaches could use it to identify patterns in round outcomes and improve economic management and site control strategies. Additionally, the dataset could support game analysts and esports commentators by providing real-time predictive insights to enhance strategic breakdowns during live matches. Analysts could highlight critical momentum shifts or economic turning points through in-game player death predictions. The dataset also holds potential applications in esports betting, particularly for live odds adjustments. Real-time statistics could be leveraged to refine in-game betting odds.

Authorized licensed use limited to: Vilnius University. Downloaded on October 22,2026 at 16:07:46 UTC from IEEE Xplore. Restrictions apply.

This would ensure dynamic adaptation to match conditions and optimize market efficiency.

VII. Conclusion

This study presents structured datasets derived from CS: GO competitive matches, designed to support predictive modeling for round outcomes and player deaths. By incorporating rich player-level and team-level features, the datasets facilitate ML applications that capture critical in-game dynamics. Our evaluation demonstrates that models such as XGBoost and Logistic Regression effectively predict round results and player survival, outperforming baseline benchmarks and prior studies. The results highlight the importance of structured feature engineering in esports analytics, with economic factors, spatial positioning, and team coordination emerging as key predictors. The dataset and models developed in this study provide practical applications for players, coaches, and analysts, enabling strategic insights, decision-support systems, and potential enhancements for training and live match commentary. Additionally, the findings could inform real-time adjustments in esports betting markets by improving in-game probability estimations.

While this study focuses on Dust 2, the methodology can be extended to other maps and game titles, broadening its impact in esports analytics. Future work could explore deep learning approaches, integrate additional contextual features, and expand predictive tasks to real-time applications, further advancing AI-driven decision-making in competitive gaming environments.

VIII. Acknowledgments

The project or effort depicted was or is sponsored by the U.S. Government under contract number W911NF-14-D-0005. The content of the information does not necessarily reflect the position or the policy of the Government, and no official endorsement should be inferred.

References

[1] D. Bednárek, M. Kruliš, J. Yaghob, and F. Zavoral, "Data preprocessing of esport game records," in *Proceedings of the 6th International Conference on Data Science, Technology and Applications*. SCITEPRESS-Science and Technology Publications, Lda, 2017, pp. 269–276.

[2] P. Xenopoulos, H. Doraiswamy, and C. Silva, "Valuing player actions in Counter-Strike: Global Offensive," in *2020 IEEE International Conference on Big Data (Big Data)*. IEEE, 2020, pp. 1283–1292.

[3] A. Björklund, W. Johansson Visuri, F. Lindevall, and P. Svensson, "Predicting the outcome of CS:GO games using machine learning," 2018, unpublished. [Online]. Available: https://odr.chalmers.se/server/api/core/bitstreams/e919ca56-3995-46b1-9546-a09b83f8b172/content

[4] P. Xenopoulos, B. Coelho, and C. Silva, "Optimal team economic decisions in Counter-Strike," 2021.

[5] A. Rubin, "Predicting round and game winners in CSGO," Preprint, Center for Open Science, Jan. 2022. [Online]. Available: http://dx.doi.org/10.31219/osf.io/u9j5g

[6] W. Xu Huang, J. Wang, and Y. Xu, "Predicting round result in Counter-Strike: Global Offensive using machine learning," in *2022 7th International Conference on Intelligent Computing and Signal Processing (ICSP)*, 2022, pp. 1685–1691.

[7] R. Paul, R. Sajeer, V. M.V, S. V. Abraham, and S. Sahajan, "A real time prediction model for round winners of Counter-Strike: Global Offensive," in *2024 3rd International Conference on Sentiment Analysis and Deep Learning (ICSADL)*, 2024, pp. 177–182.

[8] S. Marshall, P. Mavromoustakos-Blom, and P. Spronck, "Enabling real-time prediction of in-game deaths through telemetry in Counter-Strike: Global Offensive," in *Proceedings of the 17th international conference on the foundations of digital games*, 2022, pp. 1–10.

[9] A. Katona, R. Spick, V. J. Hodge, S. Demediuk, F. Block, A. Drachen, and J. A. Walker, "Time to die: Death prediction in Dota 2 using deep learning," in *2019 IEEE Conference on Games (CoG)*. IEEE, 2019, pp. 1–8.

[10] K. Chulajata, S. Wu, F. Scalzo, and E. S. Cha, "Predicting outcomes in video games with long short term memory networks," *arXiv preprint arXiv:2402.15923*, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2402.15923

[11] P. H. Swain and H. Hauska, "The decision tree classifier: Design and potential," vol. 15, no. 3, pp. 142–147, 1977.

[12] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.

[13] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, and D. Cournapeau, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[14] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery amp; Data Mining*, ser. KDD '19. ACM, Jul. 2019, p. 2623–2631.

[15] D. Jadhav, T. Bakade, S. Deshpande, O. Pethe, S. Kale, and Y. Madane, *CSGO Gaming Personality Prediction Using Big Five Factors*. Springer Nature Singapore, 2023, p. 147–158.

Authorized licensed use limited to: Vilnius University. Downloaded on October 22,2026 at 16:07:46 UTC from IEEE Xplore. Restrictions apply.